ODOC

OSS Fit Check for AI-Assisted Development

Repository Evidence + Developer Judgment



Hyeri Shin

Entrepreneurial Leader

Daegu University

Computer Science(M.S. student)

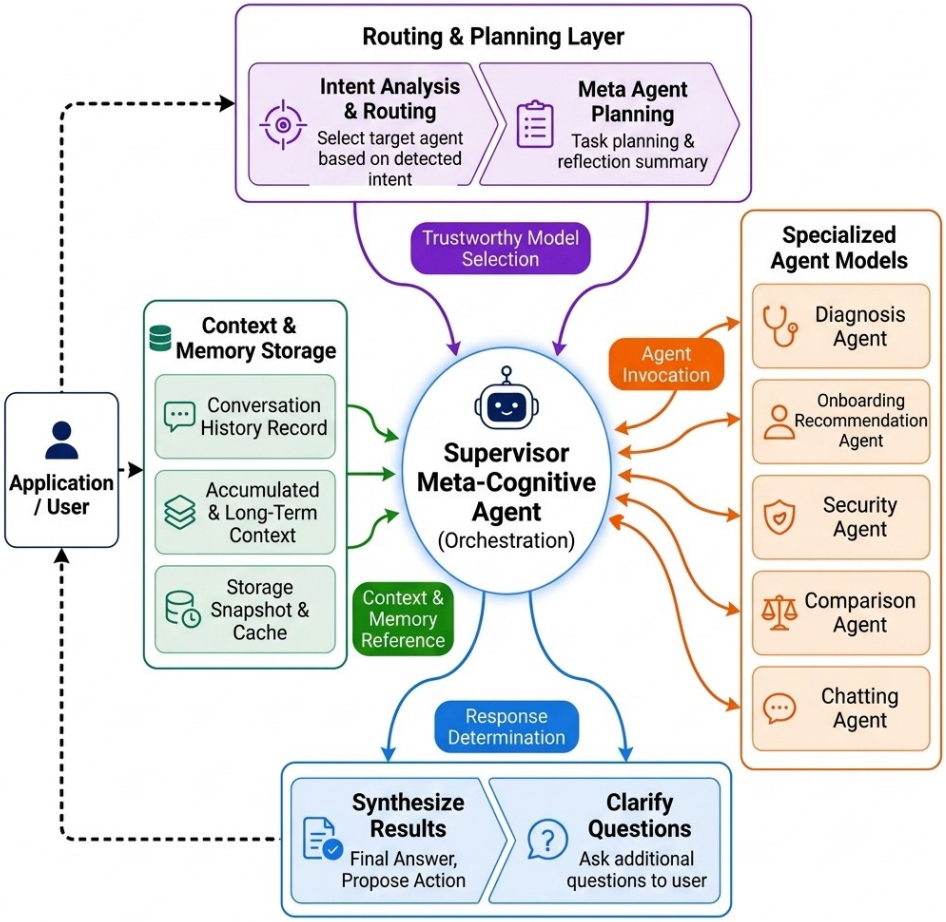


Jaeyun Jung

Entrepreneurial Member

Daegu University

Computer Science(B.S. student)



Business Thesis:
Developers increasingly use LLMs and AI agents to find OSS candidates, but they still spend time comparing candidates, checking risks, and deciding what to test first.

\ O /	Interview Count	MEET	ZOOM	CALL
NEW	21	21	0	0
TOTAL	51	50	1	0

HERD 1: From OSS Search to Candidate Comparison

01. HYPOTHESIS | This is what we thought

• Developers spend significant time manually searching and reviewing OSS candidates across GitHub, Google, and communities.

• We believed a web-based OSS Fit Check service could reduce this review burden.

02. EXPERIMENTS | This is what we did

Conducted 30 customer interviews with developers, researchers, and decision makers. We asked:

- How they found OSS candidates
- How they compared, tested, and replaced them
- What they checked before testing & what was repetitive

03. RESEARCH | This is what we learned

• Many developers already use LLMs/AI to find OSS candidates.

• Discovery is easier, but developers still struggle to:

Compare candidates, check documentation/maintenance, and decide what to test.

Core Problem: Fast & informed comparison, not just discovery.

04. DISCUSSION | This is what we will do next

Shift focus to candidate comparison and pre-screening. ODOC will:

- Compare candidates using unified scoring
- Highlight strengths/weaknesses & recommend what to test first

New Question: Which scores and risk signals most affect developers' test-first decision?

KEY LEARNING

AI reduced the search burden, but candidate comparison remains a meaningful decision problem.

HERD 2: From a Separate Web Service to In-Workflow Decision Support

01. HYPOTHESIS | This is what we thought

- We believed developers would visit a separate web service, enter OSS candidates, and review a Fit Check report before testing them.
- We assumed that individual and student developers would be our primary customers.

02. EXPERIMENTS | This is what we did

We interviewed participants who recently selected, tested, replaced, or approved OSS to understand:

- How they find and compare OSS candidates
- Where decisions are made in their development workflow
- What critical risks and information they need before testing

03. RESEARCH | This is what we learned

- Developers use LLMs/AI for discovery, but still must compare **repository health, documentation, activity, and security/compatibility risks**.
- A separate web service may add friction because many OSS decisions happen close to terminals and AI-assisted development environments.

Promising Delivery Formats: Skill, MCP, or CLI.

04. DISCUSSION | Next steps

We will test ODOC as an in-workflow decision-support prototype that compares 2–3 candidates, recommends what to test, and flags risks.

Target Audience Priority:

- Tech Leads responsible for OSS selection
- Senior Engineers comparing and integrating packages
- Engineering Managers or CTOs as buyer hypothesis

KEY STRATEGIC TRANSITIONS

Format: **Separate Web Report ➔ In-Workflow Support**
Scope: **Single Repository ➔ Multi-Candidate Comparison**
Target: **Broad Developers ➔ Tech Leads & Sr. Engineers**
Value: **More Information ➔ Faster Test-First Decision**

Critical Activities: Current Hypothesis

01

Build

Build an OSS candidate comparison engine that scores repository health, documentation, maintenance, dependency risk, license risk, and security signals.

02

Deliver

Deliver the result through Skill, MCP, or CLI so developers can compare candidates without leaving their workflow.

03

Scale

Support tech leads and engineering managers with review history, team policies, candidate shortlists, and decision records.

CURRENT ASSUMPTION

The most important activity is helping developers quickly narrow multiple OSS candidates into a ranked shortlist.

Critical Activities: What Did We Learn?

The Paradigm Shift

BEFORE

Web Report

Single-repository scoring and a separate web report with workflow friction.

AFTER

In-Workflow Comparison

Multi-candidate ranking & test-first decisions directly in work environments.

01 HYPOTHESIS

We thought the core activity was generating a web-based OSS fit report.

02 EXPERIMENT

We interviewed developers about how they search, compare, test, reject, and replace OSS candidates.

03 RESULTS (KEY INSIGHT)

The core activity should focus on faster candidate comparison, not just report generation.

ODOC should compare multiple candidates, score them, highlight weak points, and guide the next test.

04 NEXT STEPS

Build a small Skill/MCP/CLI prototype that compares 2-3 OSS candidates and recommends what to test first.

Critical Partners: Current Hypothesis

01

B2B Design Partners

Small software teams or startups that can test ODOC with real projects and real OSS candidates.

02

OSS Data Suppliers

GitHub, package registries, license databases, and vulnerability databases.

03

Potential Workflow Partners

AI coding tools, IDE ecosystems, CLI environments, and MCP-compatible platforms.

CURRENT ASSUMPTION

The most urgent partner is a B2B design partner that can provide real candidate comparison cases.

Critical Partners: What We Learned

The Partner Shift

BEFORE

Broad Networking

Relying on broad developer communities to validate product direction.

AFTER

Targeted Design Partner

Focusing on deep design partners with critical needs:

- Real projects
- Real candidates
- Real scoring feedback
- Real test outcomes

01 HYPOTHESIS

We thought broad developer communities would be enough to validate the product direction.

02 EXPERIMENT

We interviewed students, developers, researchers, managers, and people with professional OSS experience.

03 RESULTS (KEY INSIGHT)

The strongest insights came from people with real OSS adoption, replacement, or team-level responsibility.

General feedback was useful, but Real product context is essential to test whether ODOC supports actual OSS decisions.

04 DISCUSSION

We should prioritize B2B design partners who can provide real projects, real OSS candidate sets, and feedback on scoring accuracy.

Critical Resources: Current Hypothesis

01

Technology

- OSS scoring algorithm
- comparison engine
- Skill/MCP/CLI integration
- web dashboard

02

Data

- Repository metadata
- release history
- issue/PR signals
- dependency data
- license data
- vulnerability data.

03

Knowledge

- Rules for health scoring
- documentation scoring
- maintenance risk
- dependency risk
- license risk
- security risk.

04

Human Resources

- AI engineers
- backend engineers
- Customer discovery lead

Critical Resources: What Did We Learn?

The Resource Shift

EXISTING ASSETS

Current Capabilities

- Scoring algorithm
- Repository evidence analysis
- Try First / Caution / Skip logic

NEEDED NEXT

Future Requirements

Enabling deeper comparison and workflow utility:

- Real candidate comparison cases
- B2B feedback loops
- Tight workflow integration

01 HYPOTHESIS

We thought repository evidence and scoring would be useful mainly in a web report.

02 EXPERIMENT

We asked what developers check before deciding which OSS candidate to try.

03 RESULTS (KEY INSIGHT)

Our scoring algorithm is still valuable, but its best use is comparing multiple candidates and showing what candidate is weak.

Developers do not only need a score; they need a ranked shortlist and a next test plan.

04 NEXT

Use our existing scoring logic as the core resource for an in-workflow candidate comparison tool.

Cost Structure: Current Hypothesis

01

Initial Costs

- Engineering
- Integration
- B2B pilot support

02

Operating Costs

- LLM / API cost
- Data maintenance
- Customer support

01

MAJOR EXPECTED COSTS

Engineering time, Skill/MCP/CLI integration, OSS data collection, scoring maintenance, LLM/API usage, and B2B pilot support.

02

COST DRIVERS

Candidate comparison requires repository data, dependency data, license data, security signals, and repeated model or API calls.

03

CURRENT ASSUMPTION (KEY INSIGHT)

The biggest early cost is not cloud infrastructure, but engineering, workflow integration, and pilot support.

Cost Structure: What Did We Learn?

Cost Validation Status

SUPPORTED BY INTERVIEWS

Confirmed Cost Factors

- Time burden exists
- Candidate comparison matters
- Testing effort matters

UNVALIDATED

To Be Proven

- Pricing
- Willingness to pay
- CAC
- Sales cycle

01 HYPOTHESIS

We thought buyers might pay for a tool that saves OSS review time.

02 EXPERIMENT

We asked about review time, candidate comparison, replacement work, and what happens after OSS changes.

03 RESULTS (KEY INSIGHT)

OSS cost is not only review time; it includes testing, replacement, debugging, and team decisions.

However, pricing, willingness to pay, CAC, and sales cycle are still unvalidated.

04 NEXT

In the next B2B interviews, we will ask how often this happens, how many hours it takes, and which role would approve or pay for reducing that time.

Current BMC

Key Partnerships

B2B design partners with real OSS adoption cases

OSS data suppliers:
GitHub, package registries, license/vulnerability sources

workflow integration partners:
AI coding tools, CLI/MCP environments

Key Activities

Compare and score multiple OSS candidates

Highlight key risks

Recommend what to test first

Deliver via Skill / MCP / CLI

Key Resources

OSS scoring algorithm

Repository evidence and metadata

Dependency / license / security data

Engineering talent and B2B feedback

Value Propositions

Help developers compare multiple OSS candidates faster

Turn scattered repository evidence into clear scores and risk signals

Recommend which candidate to test first

Reduce wasted OSS review time before adoption

Customer Relationships

Self-service experience for developers

Pilot support for B2B teams

Feedback collection and iterative improvement

Channels

Skill / MCP / CLI inside developer workflow

Web dashboard for review history and team use

Customer Segments

Tech leads selecting OSS for production projects

Senior engineers comparing and testing candidates

Engineering managers / CTOs responsible for cost and risk

Cost Structure

Engineering and product development

Skill / MCP / CLI integration

LLM/API and infrastructure costs

Data maintenance and B2B pilot support

Revenue Streams

Paid pilot projects with small software companies

Team subscription for OSS candidate comparison and scoring

Future enterprise plans for workflow integration and dashboard use

What Will You Do Next?: Pivot

Decision: Pivot

We will pivot within the same problem space.

Keep

Core Assets & Logic

- OSS fit check
- Repository evidence
- Scoring algorithm
- Try First / Caution / Skip judgment

Change

Strategic Shifts

- **From:** Standalone web report
  To: Skill/MCP/CLI candidate comparison
- **From:**Broad individual developers
  To: Tech leads and senior engineers responsible for OSS decisions
- **From:** "Finding OSS"
  To: "Deciding what to test"

Test Next

Immediate Validation

- **Build Prototype:** Compare 2–3 OSS candidates inside developer workflow
- **B2B Testing:** Evaluate if tech leads and senior engineers make faster OSS decisions using ODOC
- **Metrics:** Measure saved time, risks found, and willingness to pay